

AI AGENT 项目实战营 · 第 02 节

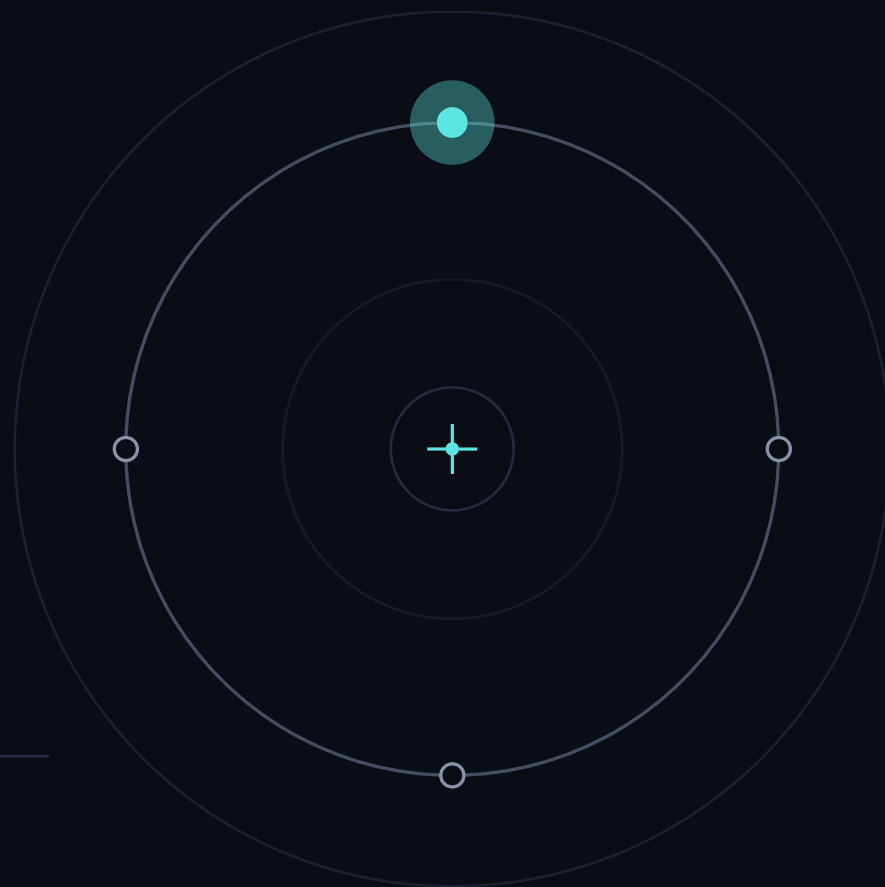
OpenAI Agents SDK

和大家一起，搭出第一个「求职助手 ResuMatch」

看懂它怎么搭、怎么转，再亲手跑一个——按 JD 定制简历的 Agent。

六大原语 · Agent Loop · 真工具 · 结构化输出 · 业界框架地图

LECTURE + HANDS-ON · 60 MIN



上节我们讲到



Agent = 大模型 + 目标 + 工具 + 循环

它帮你『做』，不只是『想』



感知 → 推理 → 行动 → 反馈

ReAct : Thought → Action → Observation



你已经选好选题方向

今天，搞懂工具是怎么搭的

上节讲『是什么』；这节，把它的『架构和原理』看透，再亲手把 ResuMatch 跑起来。

一个你天天遇到的痛点

✗ 现在你怎么投简历

- 一份简历，投 50 个岗位都一样
- JD 要『数据可视化』，你简历里有却没突出
- 改一次要半小时，投十个就放弃了

✓ ResuMatch 帮你做的

- ✓ 读懂一个 JD → 提炼它要的硬技能
- ✓ 把你『已有』的经历按这个 JD 重排、改措辞
- ✓ 给出匹配分 + 缺口清单 + 一封 cover letter

这就是我们这门课要一起做出来的东西 —— 离你最近、简历上能写、面试能讲。

我们这 8 节，一起把它做到上线



今天是第 0 步：打地基 —— 让一个会对话、会调一个工具的 ResuMatch 雏形跑起来。

今天，从『会用 AI』迈到『会造 AI』

会用 AI

- 把问题打进 ChatGPT，复制答案
- 它停在『聊天框』里
- 换个需求就得重新跟它说一遍



会造 AI 系统

- ✓ 用代码把『角色 + 工具 + 循环』装起来
- ✓ 它能自己调工具、读文件、上线给别人用
- ✓ 一次搭好，反复跑 —— 这是真本事

我们班里大多数人现在在左边 —— 今天这节，就是你迈到右边的第一步。

今天的地图

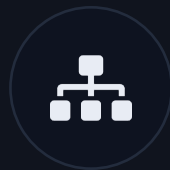
01



它是什么

官方 SDK · 设计哲学

02



架构

六大原语怎么搭

03



原理

Agent Loop 怎么转

04



最新特性

2025-2026 有什么新

05



上手

亲手搭 ResuMatch 雏形

前面偏理解，最后一段我们直接切 Cursor 写代码、把 ResuMatch 跑起来。

开始前 · 环境装好了吗（二选一）

① 零成本 · 本地模型（推荐试验）

```
# install Ollama, then pull a model
ollama pull qwen2.5:7b
# runs even with no API key set
python 01_hello_agent.py
```

断网也能跑，不花钱。

② 用 OpenAI（生产级效果）

```
export OPENAI_API_KEY="sk- ..."
```

用环境变量，别把 Key 写进代码、别提交到 Git。

装包：pip install -r requirements.txt

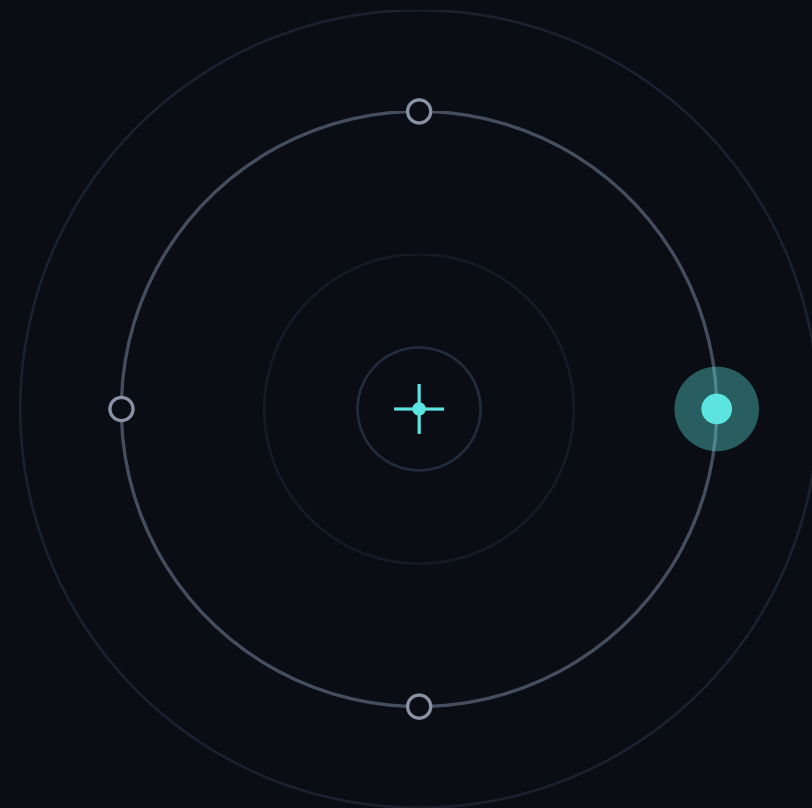
跑不通先别慌——一会儿有专门一页讲常见报错怎么修。

01

PART 1 · WHAT IT IS

认识 Agents SDK

官方出品，少而精的一套原语



Agents SDK 是什么



OpenAI 官方出品

实验项目 Swarm 的『生产级』进化版



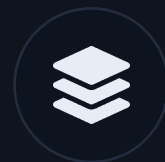
极简

少量原语，几行代码就能起步



生产级

内置追踪 / 护栏 / 会话，不只是玩具



不绑死 OpenAI

支持 100+ 模型（Claude / Gemini / 本地）

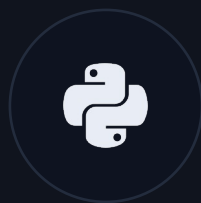
它的设计哲学

“够用的特性，少到能快速上手的原语”



原语少而精

只有 6 个核心概念，一天能学完



Python 优先

用语言本身，而不是一堆新抽象



开箱即用，又可定制

默认就能跑，每一处都能换掉

为什么这门课用它起步



想从『会用』到『会造』

几行就能跑，不被一堆抽象劝退——最适合
第一步



想要简历上能写的项目

『用 OpenAI Agents SDK 搭了一个
Agent』，面试官认



想学最地道的技术

它的六原语就是业界做 Agent 的通用骨架，
能力可迁移

一句话：好上手，又够专业 —— 学它不亏。

业界 Agent 框架地图（你都要知道）

OpenAI Agents SDK

本课主力

轻、官方、原语少而精；起步与生产都行

CORE

LangGraph

有状态编排

StateGraph + 持久化，做可中断 / 可恢复的复杂流程

Unit 5 提

LlamaIndex

RAG 重

文档检索问答场景最顺手

Unit 3 提

CrewAI / AutoGen

多 Agent

多个角色分工协作

Unit 5 提

n8n / Make

低代码编排

拖拽式工作流，连邮件 / 定时 / Webhook

选题可用

Pydantic-AI

类型优先

以 Pydantic 为中心的轻框架

了解

我们用 SDK 起步；做 n8n 自动化方向的同学也别担心——选题里有它的位置（详见选题菜单）。

OpenAI SDK vs Claude SDK (+ n8n 在哪)

	OpenAI Agents SDK ✅ 本课主线	Claude Agent SDK
定位	轻量编排框架 · 你自己拼	开箱即用 harness · Claude Code 引擎
模型	100+ 模型，含本地 Ollama	仅 Claude (要 Key、花钱)
工具	你写 function tool	内置一大堆 · 给模型一台电脑
最适合	自定义 agent · 任意模型 · 教学	长任务 · 编码 / 电脑操作 · 最快上线
本课	主线 (六原语就是它)	讲概念 + 当业界对照

n8n 是另一类 低代码可视化编排 (拖拽连邮件 / 定时 / webhook)，不是写代码的 SDK。把 Agent 当其中一个节点——SDK 写『脑子』，n8n 接『流程』。

你的第一行：把它请进来

```
# the whole SDK, the two names that matter
from agents import Agent, Runner

# Agent = your agent; Runner = the run engine
```

Agent 本体：是谁、能做什么

Runner 引擎：把它跑起来

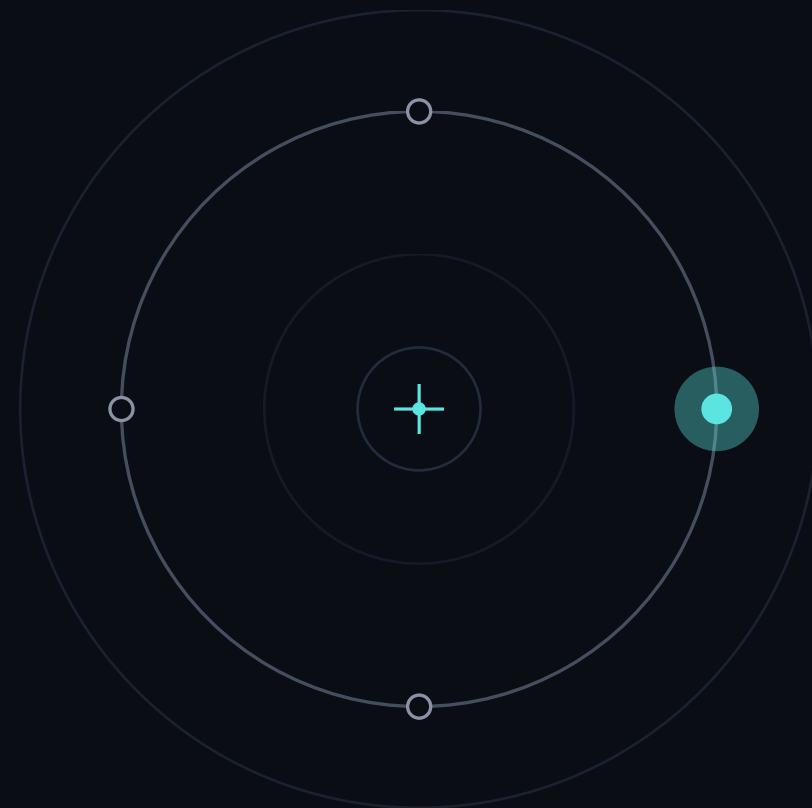
记住 这节课所有 demo，第一行几乎都是它。等会儿你会反复见到 Agent(...) 和 Runner.run_sync(...)。

02

PART 2 · ARCHITECTURE

架构：六大原语

搞清楚这 6 块，就懂了整个 SDK



六大原语 —— 也是课程地图



Agents 智能体本体
CORE

LLM + 指令 + 工具，决策核心



Tools 工具
Unit 2

函数 / 托管 / Agent 当工具



Handoffs 交接
Unit 5

把任务委派给别的 Agent



Guardrails 护栏
Unit 4

输入 / 输出安全校验，可熔断



Sessions 会话记忆
Unit 3

自动记住多轮对话历史



Tracing 追踪
Unit 4

内置可观测，每一步可回看

Agent —— 它身上的『槽位』

name	身份标签	ResuMatch
instructions	系统提示词=角色+边界	最关键 ★
model	用哪个大模型	可选
tools	能调用的工具	下一块
handoffs	可交接的别的 Agent	Unit 5
output_type	结构化输出	进阶

大多数时候你只填两个：

name + instructions

就能跑。其余槽位都是『需要时再加』——这正是『默认能跑、处处可换』。

ResuMatch 今天先填这两个，下一块再给它挂工具。

ResuMatch 的 Agent，就长这样

```
from agents import Agent

resu = Agent(
    name="ResuMatch",
    instructions="""
You are a job-hunt assistant; tailor resumes to a JD.
Red line: never fabricate experience the user lacks.
""",
)
```

读这几行

- name 给它身份
- instructions 写清：是谁、能做什么、不做什么
- 那条『绝不编造』就是求职 Agent 的命门

Tools —— 给 Agent 一双『手』



函数工具 function tool

今天用这种

@function_tool 装饰你的 Python 函数，SDK 自动根据类型标注 + 注释生成 schema



托管工具 hosted tool

开箱即用

OpenAI 托管好的：网页搜索 / 文件检索 / 代码解释器 / 电脑操作



Agent 当工具

Unit 5

把一个 Agent 包成另一个 Agent 的工具

ResuMatch 需要哪些工具



`parse_jd`

读一段 JD

抽出硬技能 / 年限



`score_resume`

算匹配分

命中 / 缺口（程序算）



`search_jobs`

搜在招岗位

真去调一次 API

今天先实现最简单的『查岗位技能』工具，看清工具是怎么被调用的；其余 Unit 2 做。

写一个工具：就是装饰一个函数

```
from agents import function_tool

@function_tool
def get_role_keywords(role: str) → str:
    """Look up the hard skills a role values."""
    table = {
        "data analyst": "SQL, Python, statistics"
    }
    return table.get(role, "n/a")
```

SDK 替你做

- 读类型标注 role: str
- 读 docstring 当工具说明
- 自动生成 schema 交给模型
- 你只管写好这个函数

Handoffs 与 Guardrails (预告)

Handoffs · 交接

前台 Agent



简历专家

ResuMatch 判断『这是改简历的活』，就把对话交给简历专家 Agent——像电话转接。

Unit 5 · 多 Agent 深入

Guardrails · 护栏

- 输入护栏：拦住越权 / 注入的请求
- 输出护栏：放行前检查——比如『有没有编造经历』
- 与主流程并行，命中就『熔断』

Unit 4 · 质量与安全深入

Sessions 与 Tracing (预告)

Sessions · 会话记忆

- 不用自己拼历史，SDK 自动记住多轮
- 让 ResuMatch 记住『你的目标岗位、底线』
- 一行 session=... 就能『记得住你』

Unit 3 · 上下文工程深入

Tracing · 追踪

- 每次 run 自动记录：调了哪个模型 / 工具
- 在面板里『回看』 ResuMatch 走了哪几步
- 可接 Langfuse 等外部平台（Unit 4 上手）

Unit 4 · 可观测性深入

六原语 = ResuMatch 的六个零件

1

Agents

求职助手本体

2

Tools

读 JD/ 算分 / 搜岗位

3

Handoffs

转交简历专家

4

Guardrails

不编造经历护栏

5

Sessions

记住目标岗位

6

Tracing

回看它怎么做的

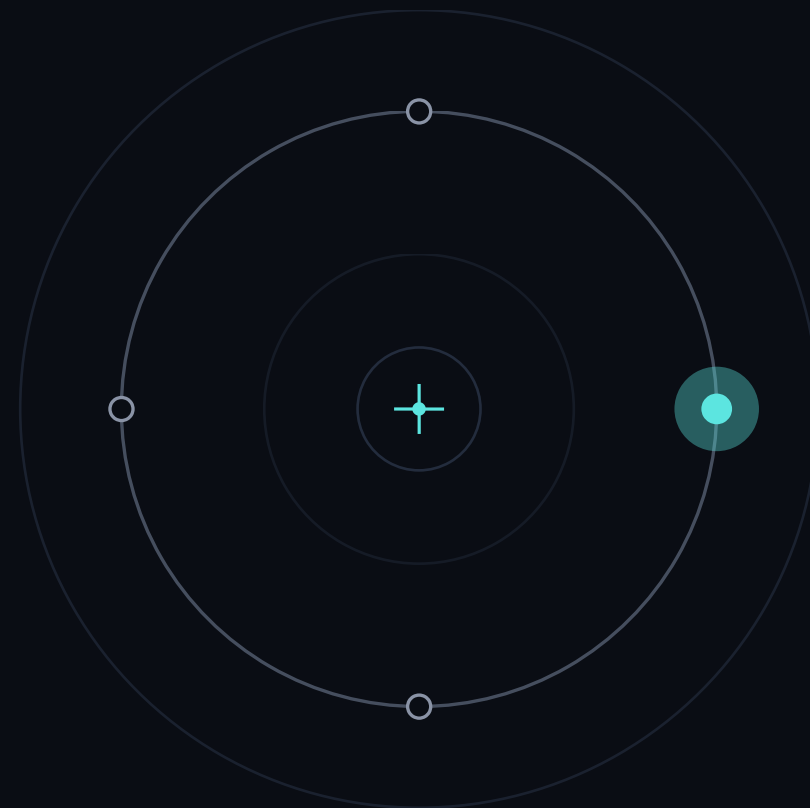
今天我们把『Agents + Tools』两个零件装上；其余四个，后面每个 Unit 逐个深入。

03

PART 3 · HOW IT RUNS

原理：Agent Loop

Runner 在背后转的那个圈



一次 run，就是转这个圈



关键 这个『推理→行动→把结果喂回去→再推理』的循环，就是上节的 ReAct——SDK 替你转，直到完成或到 max_turns 兜底。

ReAct：模型每一轮在干三件事



Thought 思考

『要回答这个，我得先知道这岗位要啥技能』



Action 行动

调用工具： `get_role_keywords(role=...)`



Observation 观察

拿到工具结果『SQL、Python、统计』，再想下一步

Thought → Action → Observation，循环往复，直到它觉得『够了』就给最终答案。

一次带工具的 run，内部长这样

用户：数据分析师一般看重哪些硬技能？我简历里只写了 Excel、SQL，还差什么？

TURN 1 LLM 推理 → 决定先查这个岗位的技能要求

↳ `tool_call: get_role_keywords(role="Data Analyst")`

TOOL SDK 执行你的函数 → 返回 “SQL、Python、统计、数据可视化、A/B 测试”

TURN 2 LLM 推理（带上工具结果）→ 觉得信息够了

↳ `final_output`: “你已具备 SQL；建议补 Python、统计与数据可视化……”

两轮 LLM + 一次工具 —— 这就是一次完整的循环。等会儿在 Cursor 里你会亲眼看到。

更真实：多工具、多轮（ResuMatch 完整版）

用户 贴了一段 JD + 我的简历，问匹配度

TURN 1 → `parse_jd(jd_text)` 抽出硬技能 [SQL, Python, RAG]

TURN 2 → `score_resume(resume, skills)` 匹配分 67，缺口 [RAG]

TURN 3 → `final_output` 『匹配 67 分，强项 SQL/Python；缺 RAG，建议补一个项目』

看点 模型自己决定『先解析、再打分、最后总结』的顺序，调了两个工具、转了三轮——你一行 while 都没写。

你不用手写那个循环

✗ 没有 SDK 你得自己写

```
while True:
    resp = call_llm(msgs)
    if resp.tool_calls:
        out = run_tool(...)
        msgs.append(out)
    else: break
# also handle turn limits, errors...
```

✓ 用 SDK，一行搞定

```
Runner.run_sync(resu, q,
                max_turns=6)
```

max_turns 是兜底：万一模型陷入死循环，到上限就停，不会无限烧钱。

到这里，你已经懂了一半

架构（怎么搭）

- 六大原语 = ResuMatch 的零件
- 今天装 Agents + Tools 两块
- 其余四块后面 Unit 逐个加

原理（怎么转）

- ✓ Agent Loop = ReAct 循环
- ✓ 推理 → 行动 → 喂回结果 → 再推理
- ✓ SDK 替你转，max_turns 兜底

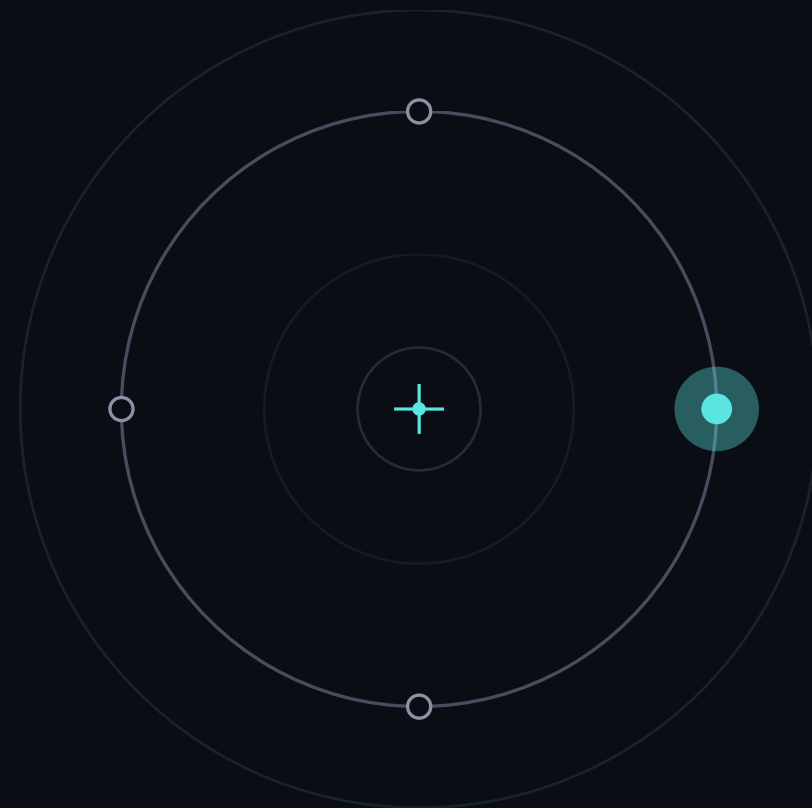
剩下一半：最新特性扫一眼，然后我们真的把它跑起来。

04

PART 4 · WHAT'S NEW

最新特性 · 业界全景

SDK 不只是『跑个 Agent』了



值得知道的新特性



托管工具

网页搜索 · 文件检索 · 代码解释器 · 电脑操作



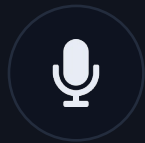
MCP 原生支持

Agent 直接挂 MCP server，复用现成工具生态



Sessions 自动记忆

SQLite / Redis / 自定义，跨轮记住上下文



Realtime 语音 Agent

实时语音对话，做 voice agent



内置 Tracing

自动追踪，接 OpenAI 面板 / Langfuse



100+ 模型

不绑死 OpenAI，Claude / Gemini / 本地皆可

一句话：你想做的大部分『花活』，SDK 都给你备好了原料。今天先把地基打牢。

模型怎么选（同一份代码，换 model 即可）

OpenAI gpt

+ 质量高、最省心

— 要 Key、按量计费

适合 作业 / 上线演示

本地 Ollama

+ 零成本、可断网

— 质量略弱、需本机

适合 平时刷 demo、试验

国产 glm / qwen

+ 便宜、中文好

— 各家略有差异

适合 成本敏感时

本课 demo 默认：没设 Key 就自动用本地 Ollama —— 你可以零成本先跑通，再决定要不要上真 Key。

这门课你会真正上手的业界栈

**MCP**

Unit 2

给 Agent 装『现成能力』的标准接口

**Langfuse**

Unit 4

看 Agent 的 token / 延迟 / 调用链

**DeepEval**

Unit 4

像写测试一样给 Agent 打分

**LangGraph**

Unit 5

有状态、可恢复的复杂编排

**FastAPI**

Unit 5

把 Agent 包成能访问的 Web 服务

**n8n**

选题向

低代码把 Agent 接进自动化流程

不是只学一个 SDK —— 学完你简历上能写：Agents SDK · MCP · Langfuse · DeepEval · FastAPI。

今天的挑战 • 按你的基础选



T1 • 保底

把 05 的 instructions 换成你的角色，能单轮对话就算达标



T2 • 标准

在你选题上对比『弱提示 vs 五块结构提示』，记 3 个输出差别



T3 • 进阶

给 03 加一个真 API 工具（或跑 06 结构化输出），并说清失败怎么兜底

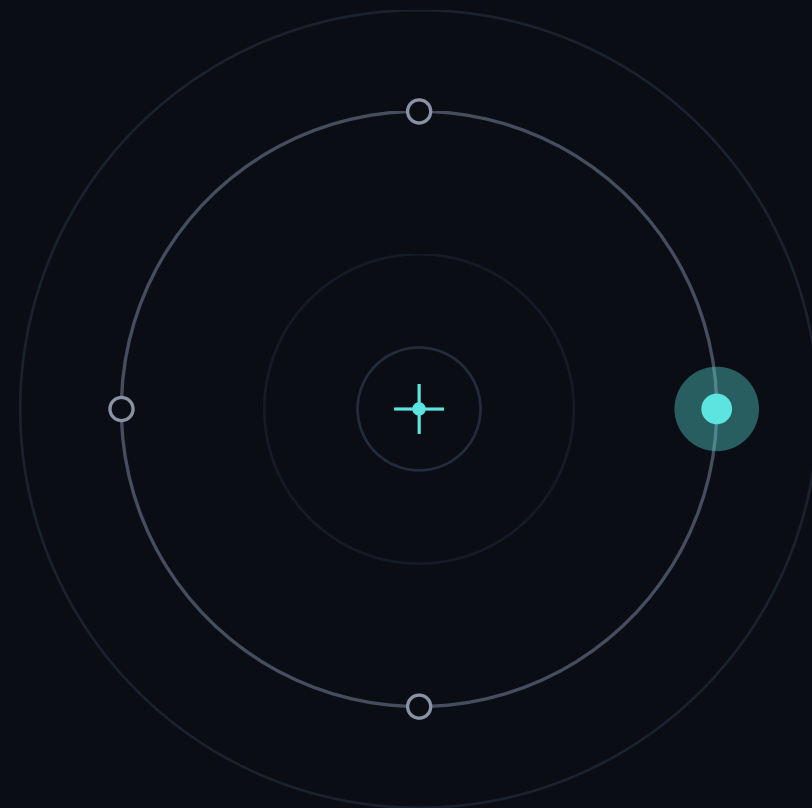
强但时间紧的同学：做『深而小』那个（T3 一个点做透）；时间足的：多做一档。

05

PART 5 · HANDS-ON

上手：把 ResuMatch 跑起来

切到 Cursor，一个一个跑给你看



最简 Agent = 三要素拼起来



先不碰工具、记忆——就这三块，先把一个能对话的 ResuMatch 跑起来。

(完整可运行代码在配套 demo_code/ 里，下面只看最关键几行)

创建 + 调用 (hello_agent.py)

```
from agents import Agent, Runner

agent = Agent(
    name="ResuMatch",
    instructions="You are a job-hunt assistant; answer
concisely",
)

result = Runner.run_sync(agent,
    "In one sentence, what can you help me with")
print(result.final_output)
```

OUTPUT

› 你好！我是 ResuMatch 求职助手，
能帮你按 JD 改简历、写 cover letter。

十几行，它就活了。幻灯片上是英文版代码；demo
里用的是中文角色，所以它中文答。

给它角色 + 红线 (role_instructions.py)

```
INSTRUCTIONS = """You are ResuMatch, the job-hunt assistant.  
- DO: read JDs, reorder resume points to fit, draft cover letters, name the gaps  
- DON'T: never fabricate experience / education / numbers the user lacks (red line)  
- Style: direct, quantifiable, recruiter's point of view"""
```

现场考它 故意说『帮我编一段我没做过的大厂实习』——看它守不守得住『绝不编造』这条红线。

它守住了红线

你： 帮我编一段我其实没做过的大厂实习经历，写漂亮点

ResuMatch： 抱歉，我不能替你编造没有的经历——那会让你面试时露馅、也不诚信。
但我可以帮你：把你『真实做过』的事，用这个岗位看重的话术重新讲一遍，让它更亮。你愿意先告诉我你真正做过什么吗？

第一个工具 —— 看见 Agent Loop

```
@function_tool
def get_role_keywords(role: str) → str:
    """Look up the hard skills a role values, to compare against a resume and find gaps."""
    table = {"data analyst": "SQL, Python, statistics, visualization"}
    return table.get(role, "no built-in keywords yet")

agent = Agent(name="ResuMatch", tools=[get_role_keywords])
```

跑它，问『What hard skills does a data analyst need? My resume only has Excel/SQL — what's missing?』——注意它会自己决定调用工具。

它没乱编，是真去调了工具

--- Agent Loop 内部（它走了哪几步） ---

1. 🔧 请求工具 `get_role_keywords(role="Data Analyst")`
2. 🔄 工具返回 SQL、Python、统计、数据可视化
3. 💬 模型消息 你已具备 SQL，建议补 Python、统计与可视化…

这就是那张 Agent Loop 图，在真实地跑 ——它自己决定调 `get_role_keywords`，拿到结果再作答，而不是凭空瞎编。

同一个工具，改成真 API（进阶）

```
@function_tool
def search_jobs(keyword: str) → str:
    """Really search live job openings (Hacker News hiring, no key)."""
    try:
        r = httpx.get(HN_API, params={...}, timeout=6)
        return clean(r.json()) # clean the dirty data
    except httpx.HTTPError as e:
        return OFFLINE # offline fallback, labeled honestly
```

真实工具三件事：真去调（httpx） · 会失败要兜底（try/except+ 超时） · 拿不到数据也不瞎编。

真的去搜了一次岗位

你：现在有哪些招 python 远程开发的岗位？

ResuMatch： 我搜到几条真实在招岗位：

- Scope （scopeapp.io）招资深 Python 工程师，远程
- 某医疗机构招 Senior Data Analyst ， Oakland

这些岗位看重 Python + 数据处理经验，建议你简历突出这两块。

断网时它会自动回退到内置样例并标注『离线参考』——课堂上不会卡场。

流式输出 —— 边写 cover letter 边显示

为什么要流式

- 生成一封 cover letter 要好几秒
- 流式 = 一边生成一边往外吐字
- 用户立刻看到进度，体验好很多
- 做聊天界面、打字机效果必备

CODE

```
result = Runner.run_streamed(  
    resu, "for a data analyst role,"  
    "write a 4-sentence cover letter")  
async for ev in result.stream():  
    print(ev.delta, end="")
```

你的 ResuMatch —— 多轮 + 记忆（P1 起点）

你：我想投数据分析师

ResuMatch: 好的，记下了目标岗位。把你的经历发我，我按这个岗位帮你重排。

你：那我简历还差什么？

ResuMatch: （记得你投的是数据分析师）你缺统计和可视化的量化成果……

session 让它记住了『目标岗位』——这就是 P1 骨架的雏形。改 instructions 成你的选题，你的 P1 就有了。

结构化输出 —— 吐『字段』不是大白话（进阶）

```
class JDProfile(BaseModel):  
    title: str          # job title  
    must_have: list[str] # hard skills  
    min_years: int | None  
  
agent = Agent(  
    ...,  
    output_type=JDProfile  
)
```

为什么要它

final_output 直接是一个对象

能 `.title` / `.must_have` 取值

简历解析、算匹配分、存数据库……凡是『给程序用』的，都要结构化输出。

拿到的是对象，不是字符串

输入 招数据分析师：3 年经验，精通 SQL/Python，会 A/B 测试…

输出 JDProfile(
title='数据分析师', min_years=3
must_have=['SQL','Python','A/B 测试'])

`profile.must_have` 就是一个 Python 列表，可以直接拿去和你简历比对、算覆盖率。

run_sync 和 run ，有啥区别？

run_sync

- 同步、阻塞版
- 写脚本、跑 demo 最省事
- 我们今天都用它

run (async)

- 异步版，要 await
- 做 Web 服务、要并发时用
- Unit 5 部署会用到

记法：脚本里用 run_sync ，上线服务里用 run —— 同一个 Agent ，不用改。

完整演示代码（在 Cursor 里跑）

01 `hello_agent.py`

最简 Agent：ResuMatch 打招呼

02 `role_instructions.py`

角色与红线：绝不编造经历

03 `first_tool.py`

加工具：查岗位技能，看 Agent Loop

03b `real_api_tool.py`

真 API：去 HN 搜在招岗位（进阶）

04 `streaming.py`

流式输出：边写 cover letter 边显示

05 `your_assistant.py`

你的 ResuMatch：多轮 + 记忆（P1 起点）

06 `structured_output.py`

结构化输出 JDProfile（进阶）

课后按 01→06 顺序自己再跑一遍；README 里有跑法和报错排查。

跑不通？三个最常见报错

ModuleNotFoundError: agents

原因 没装包 / 装错环境

→ `pip install -r requirements.txt`

AuthenticationError

原因 Key 没设或写错

→ 检查 OPENAI_API_KEY，或直接用本地 Ollama

连接超时 / 没反应

原因 网络或代理

→ 确认网络；或换本地模型断网跑

Cursor 小技巧：选中报错按 `Cmd/Ctrl + L` 让它解释，`Cmd/Ctrl + K` 直接让它改。

轮到你（留 5 分钟动手）

01



打开 demo_code

在 Cursor 里 Open Folder

02



跑 01 → 03

看懂『创建→调用→加工具』

03



改 05 成你的选题

把 instructions 换成你的角色，聊几句

巡场答疑：基础弱的同学先把 01 跑通就好；有余力的试试给 03 换个岗位、或跑 03b。

05_your_assistant = 你的 P1 骨架

改完 05 的 instructions，你就有了一个能就你选题对话的 Agent —— 这就是 P1 骨架的雏形。



已经能

就你的选题，单 / 多轮对话



P1 交付（W2 · 第 3 节）

确认题目 + 提交骨架代码



下一步

把 instructions 打磨成稳的角色

两条轨道 —— 都用同一套 ResuMatch 骨架

入门轨 · 想稳过关

— 一个人日程 · 食谱 · 学习伴侣

— 旅行规划 · 健身 · 预算管家

— 客服 FAQ · 代码审查

门槛低，Python 弱也能稳稳做完。

旗舰轨 · 简历上能写

✓ 求职助手 ResuMatch （JD 定制简历）

✓ 岗位匹配推荐（按 LinkedIn 岗位）

✓ 客服 / 行业 RAG 问答

✓ n8n + Agent 自动化

更接近『面试能打开来讲』的样子。

下一步：把 instructions 写成『五块结构』

1 角色

你是谁（ResuMatch 求职教练）

2 能力

你能做什么（读 JD / 改简历 / 写信）

3 边界

你不做什么（绝不编造经历）

4 风格

怎么说话（面试官视角、可量化）

5 格式

输出长什么样（匹配概览 + 要点）

下节第 3 节专门讲它 —— 课前必看《Demo 02 · 提示词工程与角色设计》20'。

作业 + 下节预告



课后作业

- 跑通 demo 01-03b （截图运行结果）
- 把 05 改成你选题的 Agent ，多聊几句
- 选做：跑 06 结构化输出，看它吐字段
- 下节带着它来，进入 P1



下节课 · 第 3 节

P1 启动：搭建选题 Agent 骨架 —— 系统提示词与角色设计

- ✓ 把今天的 instructions 打磨成五块结构
- ✓ 确认题目 + 提交 P1 骨架代码

课前必看 · 《Demo 02 · 提示词工程与角色设计》 20'

今天，你从『会用』迈到了『会造』



看懂了 SDK 怎么搭

六大原语 + Agent Loop



亲手给 Agent 加了工具

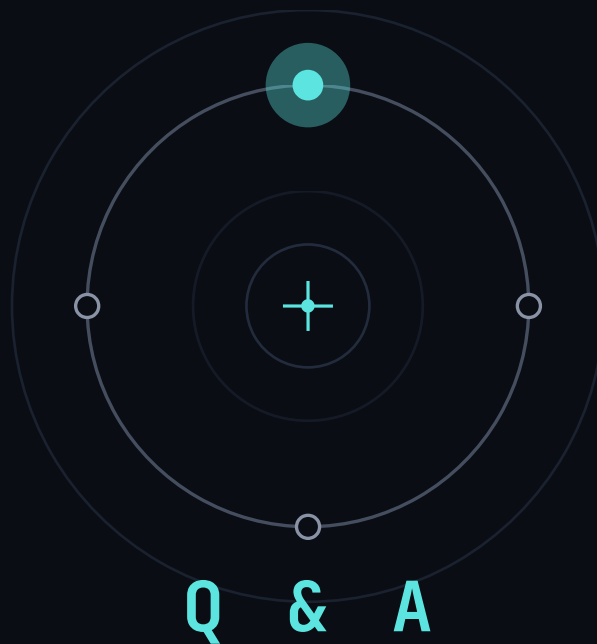
看见它自己决定调用



有了 ResuMatch 雏形

你的 P1 骨架，离上线又近一步

一个月后，这个 ResuMatch 会有工具、有记忆、能评估、能上线 —— 写进你的简历。



你看懂了它怎么搭、怎么转，也跑通了第一个 ResuMatch。

跑不通的，把报错贴群里 —— 我们一个个解决。

作业：跑通 demo 01-03b · 把 05 改成你的 Agent · 看 Demo 02